

Orama Network

A decentralized, privacy-first substrate for ordinary cloud primitives

Technical Whitepaper, First Edition · September 2026

Written by members of DeBros, who remain anonymous for the same reasons this system exists.

Written against the source, which is in alpha. Where this paper and the project's documentation disagreed, the source was read and the source won.

ABSTRACT

The commercial cloud is a single operator holding the keys to everything that runs on it. That arrangement is not incidental — visibility, control and identity all follow structurally from one company owning the machines. Self-hosting escapes it and reintroduces the operational burden the cloud removed. We describe a third position: a platform assembled from ordinary Linux services, distributed across nodes that independent operators own, and presented through one SDK and one CLI.

Orama Network implements this today. Each tenant receives a physically isolated cluster — its own consensus group, cache ring and gateway process — rather than a row prefix. All inter-node traffic crosses a WireGuard mesh that constitutes the system's single security perimeter. Authentication is a wallet signature, so no account, password or personal identifier is ever collected. Ten services are implemented and in production use by a real tenant.

We state the threat model precisely, including what it does not cover: an adversary able to snapshot the memory of a running node defeats every control described here. The engineering objective is narrower and achievable — moving secrets and plaintext out of the cheap, retroactive, offline capture path and into the expensive, targeted, live one. We report the current state without varnish, including a security audit of our own system that produced ninety findings, and we enumerate the open architectural questions that remain unresolved.

CONTENTS

PART I · WHY	PART II · THE SYSTEM	PART III · THREAT MODEL
01 The cloud is a landlord	04 Two planes, one gateway implementation	16 The adversary, stated plainly
02 What Orama is	05 One node, four tiers	17 What holds, and what does not
03 What Orama is not	06 The trust boundary is the tunnel	PART IV · OPERATING
	07 One socket, three destinations	18 How a node joins
	08 The network is its own nameserver	19 The rule that cannot be bent
	09 A namespace is a real cluster	20 A node an operator cannot log into
	10 Identity: wallet first, scopes second	PART V · WHERE THIS GOES
	11 The path a request takes	21 Honest state of the project
	12 The service surface	22 Open decisions and direction
	13 Compute: WASM and five runtimes	23 Colophon
	14 Realtime: relay by design	
	15 Storage, and what deletion means	

1 THE CLOUD IS A LANDLORD

The commercial cloud solved a real problem. Before it, shipping software meant buying machines, racking them and hiring someone to keep them alive. Now it means a credit card and an API call. Nobody sensible wants to go back.

But the trade that made it work has a second half that is rarely priced. In exchange for not owning the ma-

chines, you accept that *one company* owns them — and everything that follows. Your database sits on their disks, in a datacenter you cannot enter, under a jurisdiction you did not choose. Your traffic crosses their network, where it is logged. Your users reach you through an account system that identifies them to the provider too.

None of that is a bug in the cloud. It is the cloud. A landlord holds the keys by definition.

Figure 1. Two ways to not own the machines

ONE OPERATOR — THE HYPERSCALER	MANY OPERATORS — A NETWORK
• Visibility. The operator runs the hypervisor, the storage layer and the network. Data is plaintext when queried, in an address space they own. Encryption at rest is decrypted by a key they also hold. • Control. Accounts suspended, regions withdrawn, prices and terms rewritten unilaterally. The dependency is asymmetric. • Identity. Every service is reached through an account tied to a legal identity and a payment instrument. Anonymity is excluded at the front door. • Failure domain. One company, one set of incentives, one legal jurisdiction.	• Visibility. No single party sees the whole. Each tenant's data lives in its own processes on machines chosen from a pool, and the perimeter is a tunnel rather than a datacenter wall. • Control. No account to suspend. Nodes are replaceable and the network reconciles around a departure. • Identity. A signature, not an account. Nothing is collected, so nothing can be disclosed. • Failure domain. Many operators, many jurisdictions — which is a security property, not a marketing one.

The right-hand column is the claim, not the achievement. Sections 16 and 17 state precisely how far it currently holds and where it stops.

The usual answer to the left-hand column is self-hosting, and it does address all three rows. It also reintroduces exactly the problem the cloud solved: you are racking machines again, at one machine's worth of reliability. The honest version of that pitch is a trade of an operator you do not control for an operations burden you cannot carry.

A cloud does not have to be one company's datacenter. It can be a network of machines that many different people own, presenting one interface.

That reframing is not new — the internet already is such a network at the transport layer, and BitTorrent, Tor and IPFS each demonstrated it for storage and routing. What has not existed is the boring middle: a decentralized substrate offering the ordinary primitives an application needs. A SQL database. A cache. Somewhere to put files. A queue. Somewhere to run a function. A domain name with a certificate on it.

2 WHAT ORAMA IS

A decentralized application platform, assembled from ordinary well-understood Linux services, running on VPS nodes that independent operators own, and presented to a developer through one SDK and one CLI.

Figure 2. Three commitments, and what each forces

COMMITMENT 01	COMMITMENT 02	COMMITMENT 03
No shared substrate between tenants Multi-tenancy is physical, not a prefix on a row. A namespace is a real consensus group, a real cache ring and a real gateway process. Forces: port blocks, process supervision, a scheduler — and a blast radius that ends at the tenant boundary.	No plaintext on the public internet Exactly one security perimeter: a WireGuard mesh. Every service not deliberately public binds to an overlay address. Forces: a short auditable public port list, and a join protocol that establishes the tunnel before any service starts.	No passwords, no accounts Identity begins with a wallet signature and nothing else. No email, no password hash, no profile. Forces: a scope system, because a credential that cannot be tied to a person must carry its own authority.

Everything in Part II is downstream of these three. Where the architecture looks unusual, the cause is almost always one of them.

Every node is the same node

There is no coordinator machine, no special node type and no privileged operator. Every node runs the same software; a node differs from its peers only in flags — whether it answers DNS, whether its operator opted into running an anonymity relay, and which tenants the scheduler placed on it.

What a developer gets

Eleven services, reachable through a single TypeScript client or the orama CLI, scoped to a namespace owned by the wallet that created it. Section 12 gives the full surface; the short version is that the primitives are the familiar ones, deliberately.

The eight binaries

orama	the CLI — the largest component after the gateway
orama-node	libp2p host, database manager, mesh and swarm sync, DNS heartbeat — <i>and the control-plane gateway, embedded in-process</i>
gateway	the per-tenant data plane, one process per namespace per node
orama-sfu	selective forwarding unit for realtime media
orama-turn	relay, including the TLS and stealth listeners
orama-sni-router	the TCP ClientHello router that owns port 443
orama-inspector	SSH-based deep health checks
orama-identity	key and identity utility

Plus orama-agent, a separate module that is the entire userland on dedicated-image nodes, and vault-guardian, written in Zig.

3 WHAT ORAMA IS NOT

A whitepaper that only lists strengths is marketing. These are deliberate exclusions or honest current lim-

its, and they belong before the architecture rather than after it.

NOT THIS A blockchain Wallet signatures are used for identity because public-key cryptography is a better login than a password and collects nothing. That is the entire relationship. No chain, no consensus over application state, no token required to use the network, no contract in the request path. Consensus exists only where a database needs it — Raft, inside a tenant's own cluster, over that tenant's own rows.	NOT THIS Censorship-proof Nodes are ordinary VPS instances at ordinary hosting providers. A provider can terminate a node and a determined adversary can lean on several. What the design buys is that no single node is load-bearing: a tenant's cluster spans three, deployments replicate, and the network reconciles around a departure. That is resilience, not immunity.
NOT THIS A defence against a hostile hypervisor The most important limit in this document, and Part III is devoted to it. Orama runs on machines whose firmware and hypervisor belong to someone else. An adversary who can snapshot the memory of a running node, repeatedly and on demand, defeats every control described here.	NOT THIS Finished Orama is alpha. It runs a development network and a test network with a real tenant in production use, and it is not open for public sign-ups. Several structural questions in Part V are genuinely unresolved — including whether the current two-plane split survives at all.

ON THE WORD "WHITEPAPER"

In this industry the word has come to mean a fundraising document describing a system that does not exist yet. This one describes software that is running, and marks the parts that are not. It is not an offering document, it does not describe a token, and it makes no financial claim. Part V is the only forward-looking part; every item in it is written as an open question with its blockers, because that is what those items are.

Three claims we will not make

A claim you will not find here	Why not
"Your data is encrypted, so it is private."	Any store that answers a query holds the key and the plaintext in memory at query time. Encryption at rest addresses a disk, not a hypervisor.
"Decentralized, therefore unstoppable."	The current network is a small number of nodes run by a small number of people. Permissionless participation is deliberately out of scope until a hostile node cannot harm a tenant — see Section 22.
"Production-ready."	Section 21 lists what is not, including a security audit of our own system that produced ninety findings.

4 TWO PLANES, ONE GATEWAY IMPLEMENTATION

The single most important structural fact about Orama is that the same gateway implementation runs in two

different roles — once as the cluster-wide control plane, and once per tenant as a data plane.

Figure 3. The two planes



Read this figure first; the rest of Part II follows from it. A request to a tenant is authenticated on the core gateway — where the API keys actually live — and then proxied inward with the verified identity stamped into internal headers. The tenant's gateway never has to trust the caller; it trusts the header, and the header only arrives over the overlay.

The two roles are the same code but not the same process. The control plane is *embedded*: orama-node starts it in-process, in its own address space, which is why no core gateway systemd unit exists on a node. The data plane is that same package compiled as a standalone gateway binary and run as one template unit per tenant. A reader looking for an orama-gateway service will not find one; the name survives only in legacy cleanup paths.

Almost every other property of the system is a consequence of this split. Ports are allocated in blocks because tenants get real processes. The trust boundary has to be a VPN because those processes talk across ma-

chines. Deployments need a home node because writes must land in exactly one place.

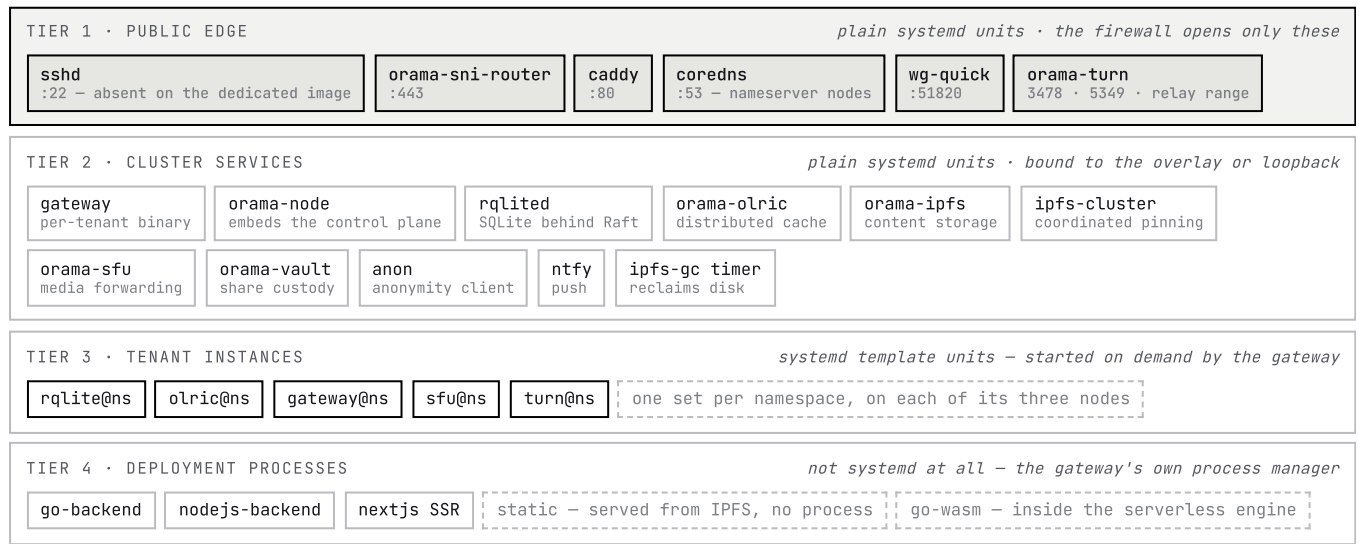
THE SEAM IS ALSO A DEFECT GENERATOR

Honesty requires noting that this split is the leading candidate for removal. Four separate defects trace directly to it: a gateway holding two database handles and silently writing tenant rows to the wrong one, API keys hashed differently by the two registries, namespace identifiers that are not portable between them, and configuration drift between the two gateway modes. Section 22 covers the proposal to collapse it.

5 ONE NODE, FOUR TIERS

A node is not special. Every node runs the same set of units, supervised in four distinct ways — and only the first tier is reachable from the internet.

Figure 4. Everything running on one machine



Four tiers, four supervision models. The differentiators between one node and another are only: the `--nameserver` flag, the operator's anonymity-relay opt-in, and which tenants the scheduler placed here. Everything runs as an unprivileged user under systemd hardening; only the web server and DNS server may bind privileged ports, because only they need to.

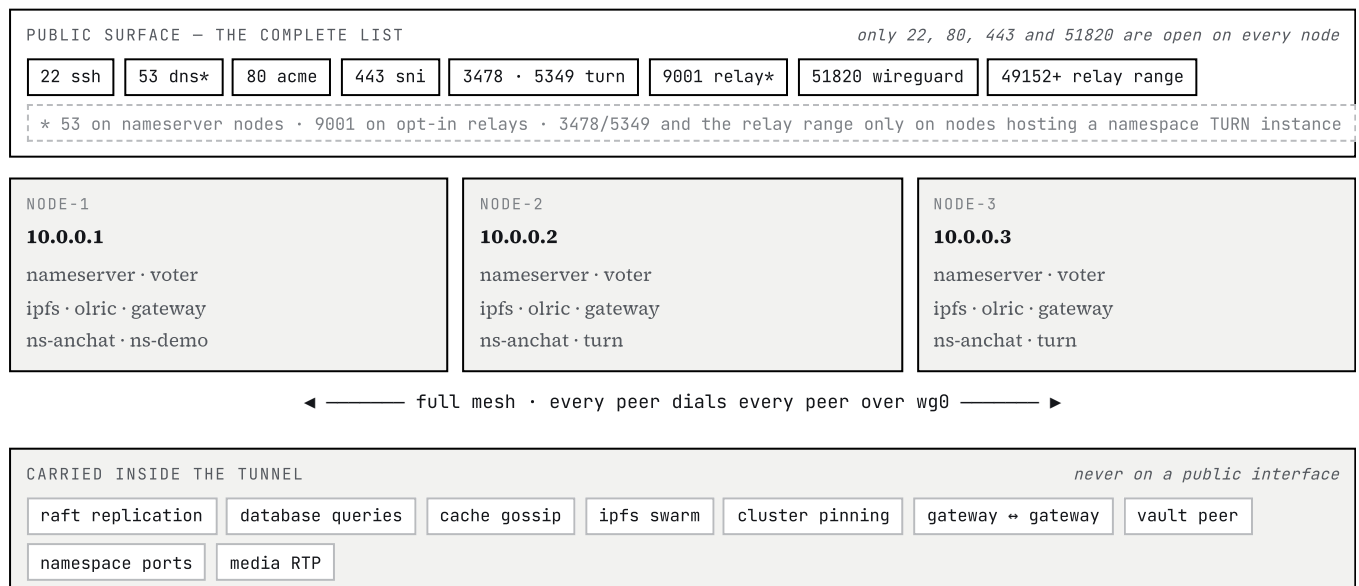
ORDERING IS A HARD DEPENDENCY, NOT A PREFERENCE

Tenant services are ordered database → cache → gateway by systemd dependencies. A gateway that starts before the cache is reachable does not fail permanently: it retries five times with exponential backoff, then keeps retrying in the background and enables its cache endpoints once the connection succeeds. Nothing in the codebase waits by sleeping – readiness is polled against the actual indicator with a timeout, because a sleep long enough on an idle machine is not long enough on a loaded one.

6 THE TRUST BOUNDARY IS THE TUNNEL

There is exactly one security perimeter in Orama and it is WireGuard. Every node holds an overlay address; every service not deliberately public binds to it.

Figure 5. The mesh, and the short list outside it



Nothing internal is exposed. Peers are exchanged over internal endpoints that require the cluster secret; the authoritative peer list is a database table, and every node reconciles its live interface against it on a loop — so a node that missed an update repairs itself rather than staying silently partitioned. IPv6 is disabled system-wide so it cannot be used to route around the IPv4 rules.

Authentication underneath the tunnel

The tunnel is the perimeter but not the only control. Each service authenticates independently, so a process

compromised inside the overlay still cannot read everything.

Figure 6. Defence in depth, and one gap

Service	Control beneath the tunnel	Enforced?
Cache gossip	Memberlist encrypted with a shared 32-byte key	yes
Content pinning	Membership gated by the cluster secret, the mesh and invite enrolment — the per-peer allowlist was deliberately removed	By other means
Internal endpoints	Cluster secret required; overlay-only	yes
WebSocket upgrades	Origin validated against the node's configured domain	yes
Vault	Session-token authentication	Partial
Cluster database	HTTP basic auth — documented, never applied	No

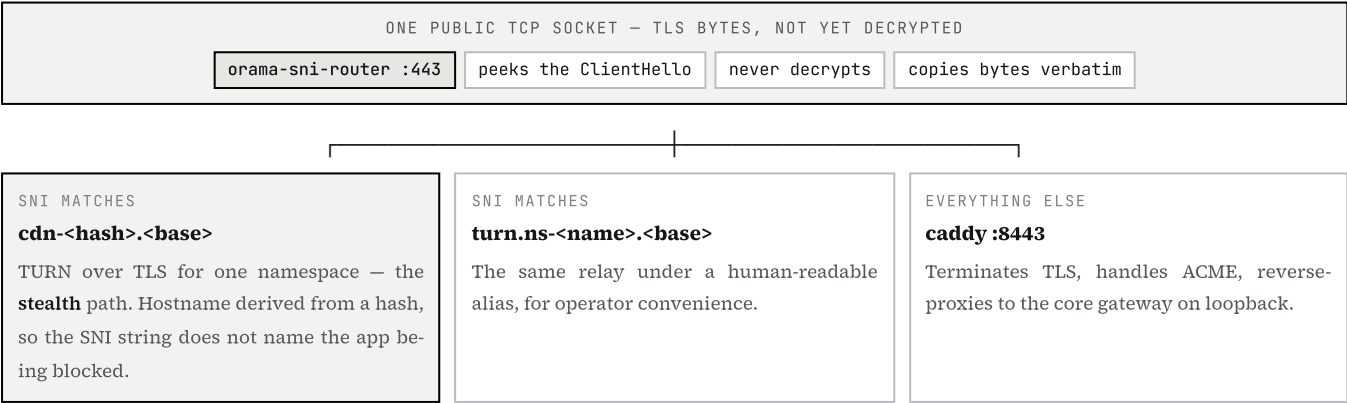
The last row is a defect we are choosing to publish. Credentials are generated and written to disk, but the configuration field that would pass them to the database process is never assigned, so the flag is never added. The firewall alone keeps the database off the public internet; anything that reaches the overlay reaches it unauthenticated. A perimeter you believe in but do not have is worse than one you know is missing.

7 ONE SOCKET, THREE DESTINATIONS

Port 443 does not belong to the HTTPS server. It belongs to a TCP proxy that reads the unencrypted TLS

ClientHello, looks at the server name, and forwards raw bytes onward without ever terminating TLS.

Figure 7. What actually owns port 443



Why bother: so that a media relay can share the HTTPS port. To a deep-packet-inspection middlebox in a jurisdiction that blocks voice traffic, a relayed call is indistinguishable from an ordinary HTTPS connection — same port, same handshake, and a hostname that looks like a CDN. Static operator routes win over auto-discovered ones on conflict, and a transient read failure keeps the previous route table rather than blackholing live traffic.

Privacy properties here are obtained by arranging ordinary components carefully, not by inventing cryptography. Nothing in that path is novel. The novelty is that it is wired up at all.

Certificates

The web server holds every certificate and serves the wildcard and apex for each zone. Certificates for tenant custom domains are issued on demand: the web server

asks the gateway whether a hostname is one it should get a certificate for, and the gateway answers from its own domain table. Challenges are answered through a small custom plugin that writes the challenge record into the cluster's own DNS — so there is no external DNS provider anywhere in the certificate path.

8 THE NETWORK IS ITS OWN NAMESERVER

Nameserver nodes run CoreDNS with a plugin that reads the cluster database. The authoritative zone is a table. There is no zone file, and no external DNS provider anywhere in the system.

Figure 8. Topology changes are database writes

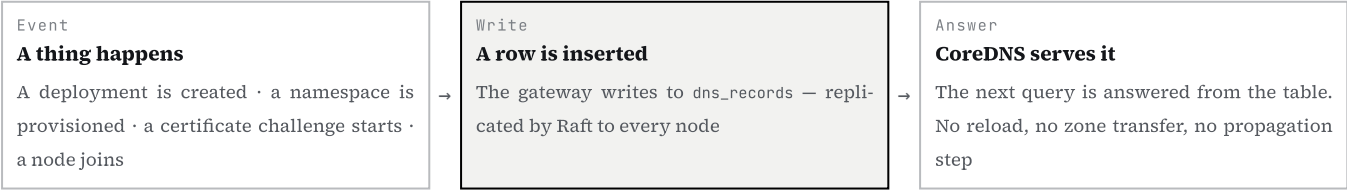


Table	Holds
dns_records	A, AAAA, CNAME and TXT for deployments, namespaces and certificate challenges
dns_nodes	Every node's peer ID, public address, overlay address, active flag, heartbeat
dns_nameservers	Which nodes answer on port 53 — the NS set
reserved_domains	Names tenants may not claim
global_deployment_subdomains	Cluster-unique subdomain reservations

Why this matters more than it sounds. Adding a node, provisioning a namespace and issuing a certificate all become the same kind of operation: an insert. There is no configuration management system in the architecture because there is no configuration to manage.

THE HEARTBEAT RUNS EVEN WHEN REGISTRATION FAILED

Each node registers itself at startup and re-asserts on a heartbeat. The heartbeat is deliberately independent of the initial registration succeeding — a node that boots during a leaderless window, which is common during a rolling restart, would otherwise stay marked inactive forever and silently drop out of both deployment routing and vault guardian discovery.

Custom domains

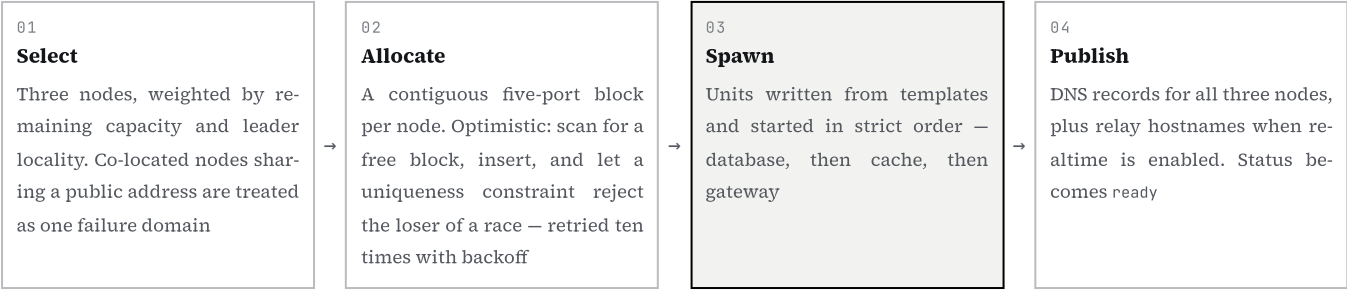
A deployment gets a generated subdomain reserved cluster-wide. A tenant may attach their own domain in-

stead: add it, prove ownership with a TXT record, point an A record at the deployment, and the certificate is provisioned automatically. Ownership is verified before a certificate is requested, so the platform cannot be used to obtain certificates for domains the requester does not control.

9 A NAMESPACE IS A REAL CLUSTER

In most platforms a tenant is a column. Here a tenant is a cluster — its own consensus group, its own cache ring, its own gateway process, on three real machines.

Figure 9. Provisioning, and why it is recoverable



Every step writes an audit row — twenty-seven declared event types. That trail is what lets an operator see exactly which node reached which service after a failed provision. It is an audit log rather than an event source: authoritative state lives in mutable rows updated in place, and the trail is deleted when the namespace is deprovisioned. The other outcomes are degraded, failed and deprovisioning.

Figure 10. Port arithmetic — capacity is a division problem



Four disjoint ranges, three allocators. Realtime uses entirely separate ranges so that adding voice to a namespace cannot disturb an existing core allocation. Allocation is optimistic rather than transactional — scan, insert, and let a uniqueness constraint reject the loser of a race, which retries with backoff. The twenty-tenant ceiling is a limit *per machine*, not on the network: growth is horizontal.

The tenant's tables live in the tenant's own database file, on the tenant's own replication log, reached through the tenant's own gateway on the tenant's own port. There is no query that can cross the boundary because there is no shared table to cross it in.

10 IDENTITY: WALLET FIRST, SCOPES SECOND

Orama has no passwords. Identity begins with a wallet signature and everything downstream derives from it.

Figure 11. The credential ladder

START	A wallet EVM or Solana. Request a single-use nonce, sign it, submit the signature. The server recovers the address and binds it to a namespace.
↓ ISSUES	A short-lived token · and a scoped API key Refresh tokens are stored as hashes, never plaintext. API keys are stored as HMAC hashes under a server-side secret. Both carry a grant set.
↓ EXCHANGE	A key may be traded for a token — keeping the key's narrow grants This is the guard that closes exchange-then-escalate. A runtime key cannot be laundered into an administrative token by trading it in.
↓ GATE	The scope gate — required grant per method and path Runs <i>after</i> ownership, so it can only ever tighten an already-authorized request. It never authorizes on its own.

Two more guards make it hold. A wallet token is never trusted for a scope claim it happens to carry, even if a tenant's own claims provider injected one — the field is reserved and ignored on wallet subjects regardless. And a logged-in end user receives every data-plane grant and never administrative access — unless that wallet is a confirmed *owner* of the namespace, in which case the middleware marks the request owner-confirmed and the gate grants it admin. Ownership, not the token type, is what opens the control plane.

Figure 12. Grants by profile — what a client bundle may safely ship

Grant	admin key	app-runtime key	invoke-only key	user token
admin — deploy, secrets, migrations, key management, raw database	✓	✗	✗	✗
invoke — function invocation	✓	✓	✓	✓
storage — upload, pin, get, unpin	✓	✓	✗	✓
push — device registration, never send	✓	✓	✗	✓
webrtc — relay credentials, signaling, rooms	✓	✓	✗	✓
proxy — anonymous outbound HTTP	✓	✓	✗	✓
pubsub · cache — messaging and cache REST	✓	✗	✗	✓

Eight grants are defined, but only seven can currently be minted onto a key: pubsub is missing from the issuance allow-list, so a request for it is rejected as unknown — by an error message that lists it as valid. A pubsub route is therefore reachable today only by a user token or an admin key.

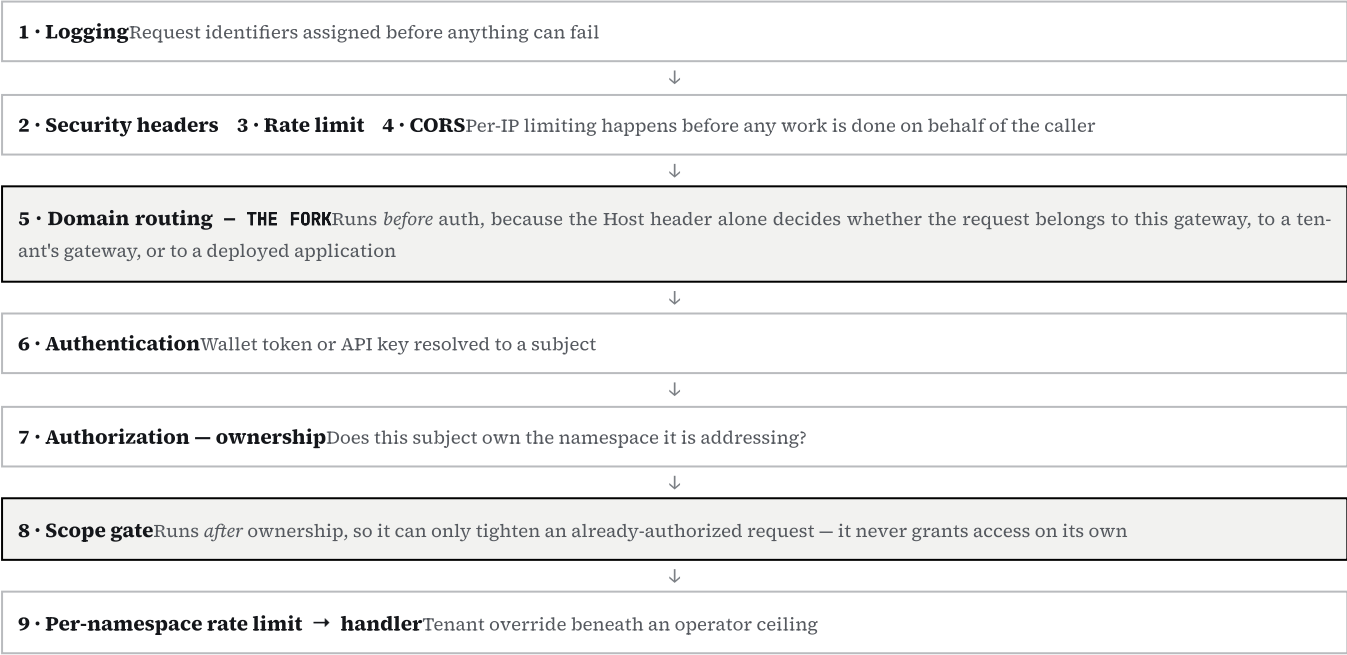
Administrative access is a wildcard that satisfies every requirement. The only thing standing between a leaked client key and the control plane is that the key was never minted with that grant — which is why the profiles exist. Keys minted before scoping existed stored a null scope, read as administrative; a grandfather branch kept them working across a rolling upgrade and a sweep then revoked them so the branch stopped mattering.

11 THE PATH A REQUEST TAKES

Every request that survives the edge lands on one multiplexer wrapped in a fixed nine-layer chain. The order is

deliberate, and one ordering decision in it is load-bearing.

Figure 13. Nine layers, then a handler



Domain routing before auth, scope after ownership. The first ordering is required because a request for a deployed application is not addressed to the gateway at all. The second is a safety property: a gate that runs after authorization can only ever subtract.

ONE DELIBERATE EXCEPTION

A host of the form ns-<name>.<base> is proxied inward regardless of path — except for scoped key management, which operates on the cluster database where keys are validated. Namespace gateways have isolated databases without the au-

thoritative key table, so proxying those writes would land keys in a database that can never authenticate them.

Reads are everywhere, writes are somewhere

Any node can answer a request for a deployment. State-changing operations — update, delete, rollback, logs, statistics — are funnelled to that deployment's *home node*,

so two gateways never mutate the same application concurrently. The forwarding proxy is loop-guarded by a header: a request already forwarded once is handled locally rather than bounced again.

Figure 14. What is offered, and what it is made of

REPLACES RDS / AURORA Database SQLite behind Raft, replicated across the tenant's own three nodes. Bounded-staleness reads reject a lagging follower rather than silently serving it.	REPLACES ELASTICACHE Cache Distributed maps with TTL and replication; members discover each other over encrypted gossip. Routes register even when the cache is not yet up, and reconnect.
REPLACES S3 / R2 Storage Content-addressed storage on a private swarm — a shared swarm key means nodes peer only with each other, never the public network. Ownership tracked per namespace.	REPLACES SNS / SQS PubSub Gossip-based messaging over WebSocket and REST, with presence and a per-namespace sequence so a batch publish can be ordered by consumers.
REPLACES LAMBDA Functions WebAssembly executed in-process — no containers, no cold-start virtual machine. A module pool removed the per-invocation startup cost.	REPLACES VERCEL / FLY Deployments Five deployable runtimes. Static served from content storage; SSR, Go and Node.js as supervised processes. Rollback is first-class, not a redeploy.
REPLACES ROUTE 53 DNS & domains CoreDNS reading a SQL table as its zone. Automatic certificates; custom domains verified by ownership proof first.	REPLACES TWILIO / DAILY WebRTC Selective forwarding and relay in server-enforced relay-only mode, with a TLS listener that shares the HTTPS port.
REPLACES SNS MOBILE Push Tenants supply their own credentials: direct to Apple, or a self-hosted server on the node for Android without Google. Never a shared platform account.	NO CLOUD EQUIVALENT Anonymity proxy Outbound HTTP through a relay network, callable from application code or a WebAssembly function. Private addresses bypass the circuit.
— SQLite Per-namespace database files with content-addressed backups, separate from the tenant's replicated database.	REPLACES SECRETS MANAGER Vault NOT READY Shamir shares across guardians on separate nodes. Shipping on every node — but see the note below before relying on it.

Eleven services; ten of them usable. The twelfth box is included because omitting it would be the more misleading choice.

THE VAULT: STATED PLAINLY

The design is sound — the client, not the server, splits a secret, and fewer than a threshold of guardians reveals zero information regardless of computing power. The defects are in the implementation. Guardian discovery is a stub: it returns an empty node list and reports success, so the self-only fallback never fires and a guardian starts knowing no peers. The newer named-secrets API authenticates with a session token that any caller knowing the identity can mint, because the challenge is the server's own value replayed back with no proof of key possession — the older push and pull paths do require a real signature. And the node agent carries a fourth, separate implementation of the sharing scheme that infers each share's position from where it sits in the collected slice, so skipping one un-

12 THE SERVICE SURFACE

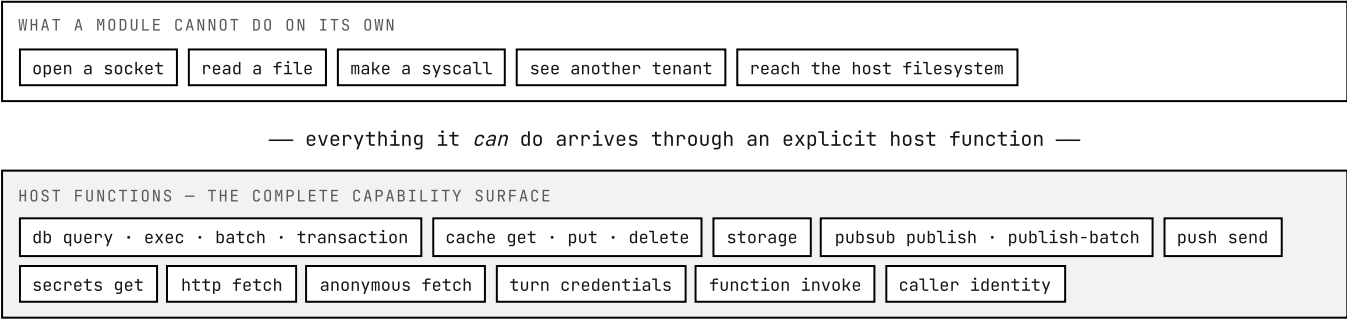
Eleven services, one client, one CLI. The label on each box states the *shape* of the service — not a claim of parity with providers that have a decade of hardening behind them.

reachable guardian silently reconstructs the wrong key. It is not a service to trust with a secret today.

13 COMPUTE: WASM, AND FIVE DEPLOYABLE RUNTIMES

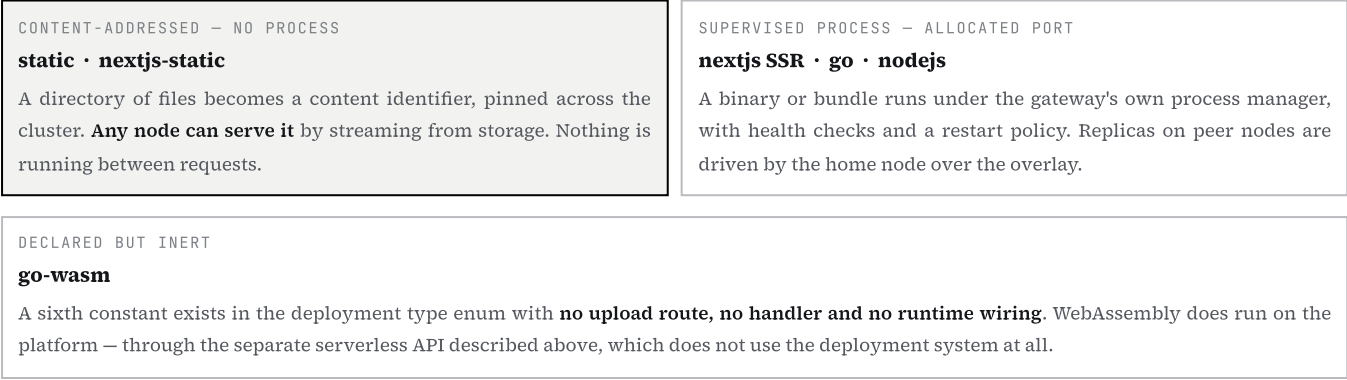
Serverless on Orama is WebAssembly executed in-process. The security property is unusually legible: capability is a list you can read.

Figure 15. The sandbox — the host function table is the policy



Capability is enumerable. Auditing what tenant code can reach is reading one table rather than reasoning about a container escape. The known gap: outbound HTTP currently has no address filter, so a function can reach node-local services — a tracked defect, listed in Section 21.

Figure 16. Five deployable runtimes, two fates



The Host header decides everything. A static deployment never reaches a process; a dynamic one is proxied to a local replica; anything that changes state is forwarded once to the node that owns the deployment. The two subsystems are separate: deployments and functions share neither a route nor a type.

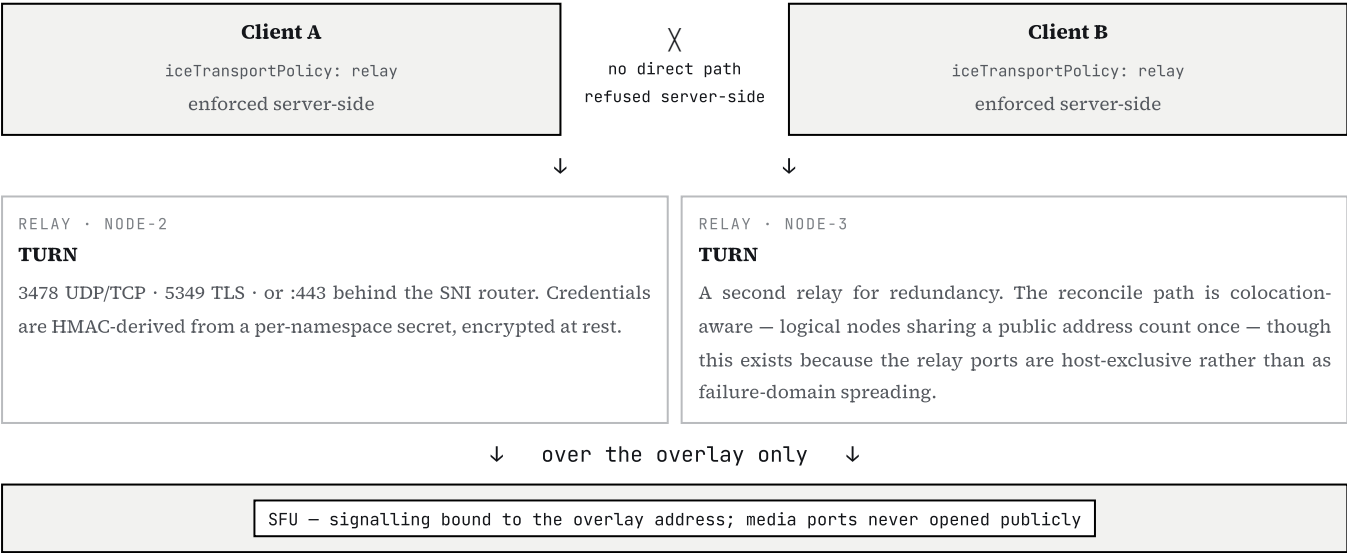
RECURSION IS BOUNDED, NOT FORBIDDEN

A function can publish a message, and a published message can wake a function. Dispatch therefore carries an explicit depth parameter, and nested chains are bounded; the local fast path is subject to the same depth accounting. Once-per-publish dedup — an in-process cache plus a cluster-wide claim — guards the shared dispatch entry point, while the local path fires only wildcard-pattern matches on the publishing gateway itself.

14 REALTIME: RELAY BY DESIGN

A namespace can opt into voice, video and data channels. The security model is the unusual part: relay is not a fallback here, it is the only path.

Figure 17. Forced relay — peers never learn each other's addresses



Clients cannot reach the forwarding unit directly, so all media must traverse a relay, and the server enforces relay-only transport so that no direct connection is even attempted. The consequence is that two people on a call never learn each other's addresses. Combined with the TLS listener sharing port 443, a call is both unlinkable between participants and, to an on-path observer, not obviously a call. **Stated precisely:** the signalling listener is configured to the overlay address by provisioning convention rather than by a validated invariant, and the media sockets are not interface-bound at all — what keeps the unit off the public internet is the firewall, which never opens its ports.

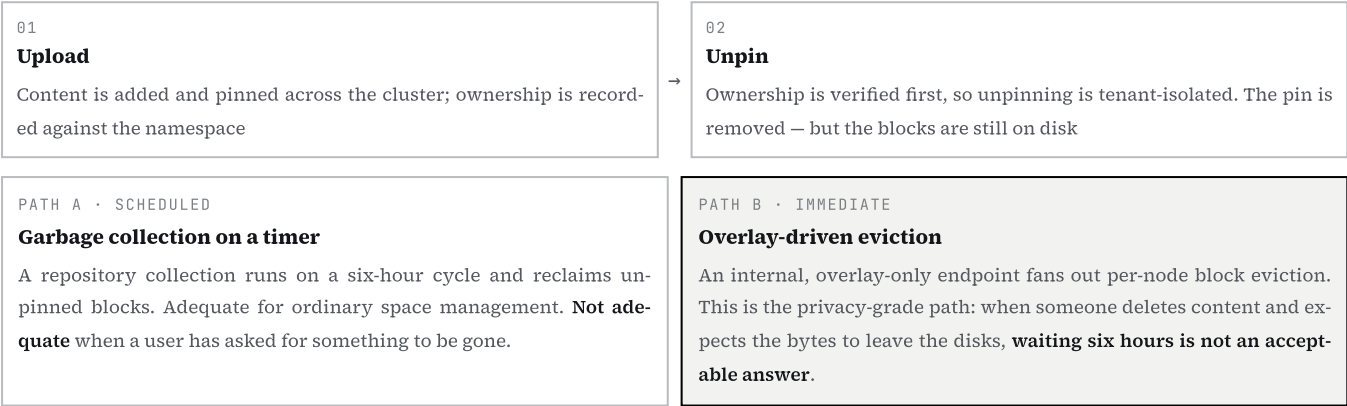
A GAP THAT WAS REAL, AND IS CLOSED

Node replacement used to leave realtime role allocations attached to the departed machine — one cause behind three visible symptoms: a namespace losing relay redundancy, crash loops on replacement nodes, and orphaned allocations. It is fixed. A reconciler now runs on a sixty-second loop, revoking roles held by members that are no longer viable and re-allocating to live ones, under a majority quorum guard and a single deterministically elected coordinator. The residual is review follow-ups on that fix, not a missing migration.

15 STORAGE, AND WHAT DELETION MEANS

Content-addressed storage is straightforward until a user asks for something to be deleted. Then the question is whether the bytes actually left the disks, and the honest answer requires more than an unpin.

Figure 18. Two reclaim paths, and why both exist



Why the distinction is in a whitepaper at all: because "delete" meaning "eventually, probably" is the default behaviour of content-addressed storage, and a platform that markets privacy has to close that gap explicitly rather than inherit it.

Capacity is derived, not fixed

The storage ceiling on a node is a share of that node's actual disk rather than a constant, so a larger machine contributes more without reconfiguration. A swarm-sync loop keeps every node connected to every cluster peer, so a node that restarts cannot silently end up isolated while its daemon still reports healthy. Per-name-

space byte quotas are checked server-side before every upload and pin, with a replication-aware projection — but they are *opt-in and currently dormant*: a namespace with no quota row is treated as unlimited, nothing yet writes that row, and a lookup error fails open.

THE LIMIT OF "DELETED"

Even immediate eviction does not erase history everywhere. A deletion is itself a replicated log entry, so the original write — with its payload — persists in the replication log and its snapshots until compaction, which is driven by size and count thresholds rather than by any privacy requirement. Deleting is not erasing, and a system that claims otherwise is not describing its own storage layer accurately.

16 THE ADVERSARY, STATED PLAINLY

Most platforms describe their security posture in terms of what they encrypt. That is the wrong frame, because

it invites the reader to assume a threat model that was never stated. This section states one.

The adversary Orama designs against is **the hosting provider**, or an agency with legal or covert access to one. That adversary can take a full disk snapshot, take a full memory snapshot at the hypervisor, boot a clone of any disk image, and observe all datacenter traffic. We control neither the hypervisor nor the firmware, and confidential computing is not purchasable on budget hosting.

Figure 19. The conversion this design buys

CHEAP PATH — WHAT WE MOVE THINGS OUT OF	EXPENSIVE PATH — WHAT WE MOVE THINGS INTO
• Retroactive. A backup taken months ago still yields today's secrets. • Scales to everyone at once. One API call captures every customer on the host. • Deniable and traceless. Nobody has to decide to target you specifically. • Asynchronous. Grab the image now, read it whenever.	• Targeted. Requires a decision about a named machine. • Synchronous. The process has to be running at the moment of capture. • Repeated. Once is not enough; keys rotate and sessions end. • Human-visible. Somebody has to authorise it, and keep authorising it.

That conversion is the entire product of the privacy work. It is not nothing — but it is also not invisibility, and it should never be sold as such. Nothing here makes capture impossible, and any claim otherwise about software running on rented machines is false.

Adversary capability	Cheap / one-shot?	Addressed by anything here?
Full disk snapshot of a running node	Yes — one API call	Yes — by moving plaintext off disk
Backup image or decommissioned disk	Yes — retroactive	Yes — same, plus retention limits
Boot a clone of a disk image	Yes	<i>Partially</i> — with real caveats
Full memory snapshot of a running node	No — targeted, timed	No. Only data minimisation and end-to-end encryption survive it
Observe all datacenter traffic	Yes	<i>Partially</i> — the tunnel covers content; topology and timing leak permanently
Compel the operator	—	No.
Control the hypervisor or firmware	—	No. This is why hardware-sealed credential schemes are rejected rather than adopted

THE CURRENT STATE, WITHOUT EUPHEMISM

A dedicated security review of this codebase — one that assumes the hosting provider, not a remote attacker, is the adversary — produced ninety findings, twenty-seven of them critical, and was independently re-verified before being filed as seven pending workstreams. Measured against today's baseline, where the cluster secret, the storage swarm key, the key-hashing secret and the tunnel private key all sit in plaintext on disk, its conclusion is that one disk snapshot of any single node is a total, permanent, offline compromise — including from a backup taken months ago or a disk decommissioned last year. Publishing the number is the point: a platform asking to be trusted with private data should show its audit, not its logo.

17 WHAT HOLDS, AND WHAT DOES NOT

Figure 20. Property, mechanism, and the attack that still wins

Property	Mechanism	Survives disk capture	Survives memory capture
No identity to disclose	Wallet signature — nothing is collected	✓	✓
Caller address hidden from third parties	Anonymity relay network, per request	✓	<i>partial</i>
Participants hidden from each other	Server-enforced relay-only transport	✓	<i>partial</i>
Call not obviously a call	TLS relay behind the SNI router on 443	✓	✓
Internal traffic not public	Encrypted mesh; overlay-bound services	✓	✗
Tenant cannot read tenant	Separate database, cache and gateway processes	<i>partial</i>	✗
Deletion reclaims disk	Immediate overlay-driven eviction	<i>partial</i>	✗
Data confidential at rest	Encryption at the value level	✓	✗

The right-hand column is the honest one. Only properties that are absences — a credential never collected, a hostname that reveals nothing — survive an adversary reading process memory. Everything achieved by encryption fails there, because a store answering a query holds the key and the plaintext at query time.

The strongest property is an absence

Authentication is a signature. There is no email address, no password hash, no phone number and no profile — not because they are protected but because they are never collected. A demand served on an operator for a user's

identity returns a public key, because a public key is all that exists. This costs nothing to operate, because it is an absence rather than a mechanism, and it is the only property in the table that no capture defeats.

Three things that are not defended

Memory A database answering a query holds key and plaintext in memory. True of every store, be-spoke or not. Rewriting the storage engine would move plaintext from one address space to another that the same hypervisor reads.	Metadata Which nodes talk to which, how often, in what volume, when. The tunnel does not hide it. Value-level encryption also leaves residue in log structures and write-ahead files. Metadata is the price.	Compulsion An operator who can be compelled can be compelled. Splitting secrets raises the number of parties who must be — a real improvement in a jurisdictionally diverse network, and none at all if every node sits with one provider in one country.
--	--	---

WHY PUBLISH THE LIMITS

Because a user who believes a system defends against memory capture will make decisions — about what to store, and about whom to store it for — that the system cannot support. Overstating a threat model does not protect anyone; it transfers risk to the people least able to evaluate it. It also has a direct architectural consequence: **operator diversity is a security property**, not a marketing one, and it is the property the network most needs to grow.

18 HOW A NODE JOINS

There are no shared secrets on a command line. A node joins with a single-use, time-limited invite, and the tunnel is established before any cluster service starts.

Figure 21. The join sequence

01 · MINT	An operator mints an invite on an existing node A row lands in the invite table and the CLI prints the full join command. The token is single-use and time-limited.
02 · CALL	The new node calls the join endpoint over HTTPS With certificate pinning on first use. Never on the gateway's own port — the firewall blocks it, and that mistake is the most common failed install.
03 · CONSUME	The handler validates and <i>atomically</i> consumes the token It allocates an overlay address and returns the cluster secret, database credentials, cache key, storage swarm key and the current peer set.
04 · PEER	The joiner brings up its tunnel and registers its public key Every existing node reconciles the new peer from the authoritative peer table — no manual configuration on any other machine.
05 · START	Only now do database, cache, storage and gateway start All of them dialling peers on overlay addresses, never public ones.
06 · ANNOUNCE	The node registers in the DNS table and begins its heartbeat Its lifecycle state moves from joining to active, and metadata is republished to the discovery layer.

The ordering is the security property. No cluster service is ever reachable before the tunnel exists, and no secret ever travels outside it — with one exception noted below, which is a defect rather than a design.

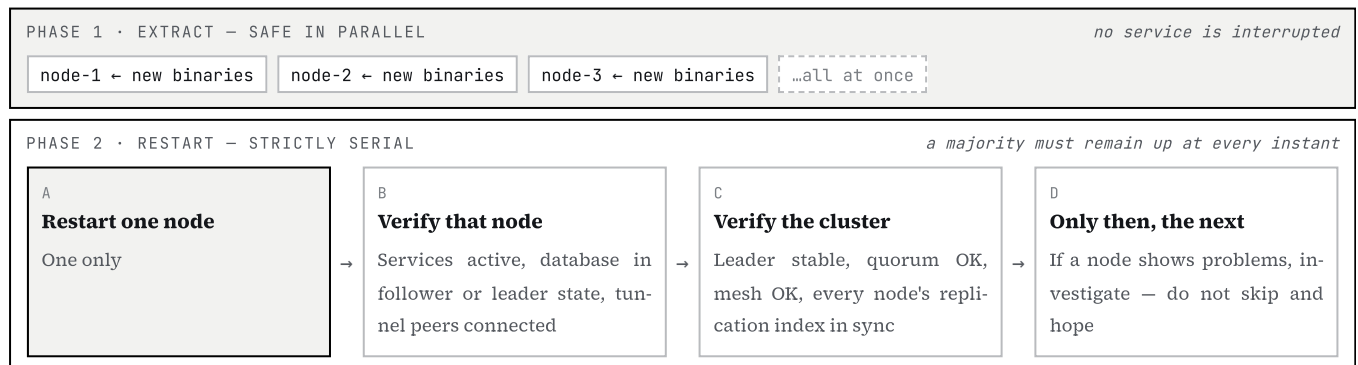
NOT YET TRUE FOR THE DEDICATED IMAGE

The flow above describes the standard install. The enrollment path used by the dedicated node image is currently weaker: its completion endpoint is unauthenticated, and it ships the node's own tunnel private key and the cluster secret over plaintext HTTP. Rewriting that wire protocol is tracked work, and until it lands the network cannot safely admit a node operated by someone unknown — which is precisely the property a decentralized network needs.

19 THE RULE THAT CANNOT BE BENT

The control plane is a Raft cluster, and Raft needs a majority of voters. Everything about how Orama is operated follows from that one sentence.

Figure 22. Extraction is parallel; restart is not



Losing a majority simultaneously loses the cluster. This is stated in a whitepaper rather than left to a runbook because it constrains the architecture: any future change that makes restarts more frequent, or that increases the number of quorum-holding services, is paying a real cost that has to be justified.

Verification against the real thing

A lesson worth recording: the cluster database is not a local SQLite file, and behaviour that passes against one can still fail on the cluster. A row-eviction defect shipped exactly that way — correct against every local test, wrong in production. Live verification against a real cluster, plus reading the gateway logs, is the only sufficient test for storage-layer changes.

TWO HANDLES, ONE SILENT FAILURE

The gateway holds a handle bound to the cluster's own database and a handle bound to a namespace's. Namespace-scoped data written through the wrong one lands in a database it will never be read back from — and nothing errors. It fails silently, which is the worst failure mode available, and it is the clearest example of the hazard the two-plane architecture creates.

20 A NODE AN OPERATOR CANNOT LOG INTO

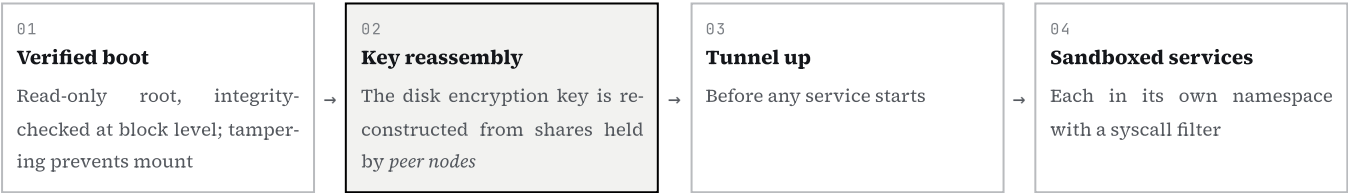
A purpose-built node image exists: no SSH, no shell, a read-only root filesystem verified at the block level, an

encrypted data partition, and exactly one root process that is the entire userland. It has never been run.

READ THIS BEFORE THE REST OF THE SECTION

What follows is a design that is built but untested. The image has not been booted in anger; its agent has no test files; integrity verification is compiled but not wired into boot; the syscall filter is never loaded into any process; and a signature-verification defect means no over-the-air update can currently be applied in the field. The present proposal is to **exclude it from the first release** and run hardened Ubuntu instead. It is described here because it is the direction, not because it is the status.

Figure 23. The intended boot chain



The intent: no path from physical possession of a machine to reading its data — the disk encrypted, the key existing only as shares on other machines, and no shell in which to ask for it. A node removed from the network cannot be booted in isolation either. **Stages 01 and 04 are not yet wired up in the shipped image.**

The design claim, and its honest limit

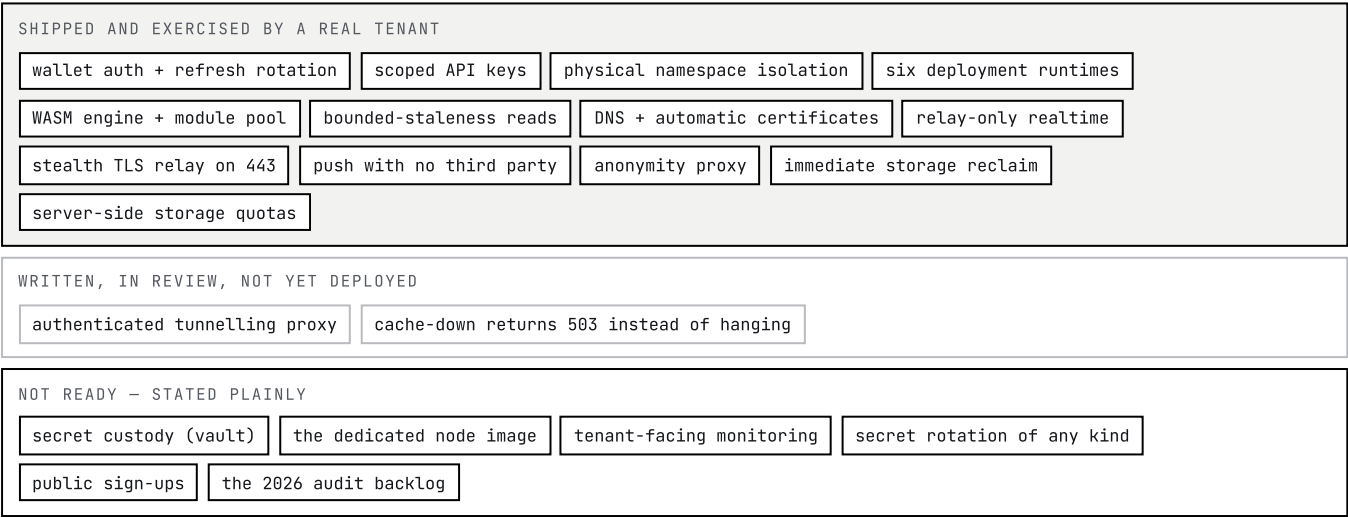
Removing the shell removes the most common path from operator to tenant data, and encrypting the disk with a key the operator never holds removes the offline one. Neither removes the hypervisor. An operator who is also the hosting provider — or who runs the image on hardware they control at the firmware level — is outside what this defends.

The sharper research question, and the one currently on the table, is not which distribution to fork but *what does this platform require that an existing minimal, API-driven, shell-less operating system does not already provide?* Forking one means inheriting a security-update treadmill indefinitely, which for a small team is the thing most likely to kill the project.

21 HONEST STATE OF THE PROJECT

Everything up to this point describes software that runs. This part describes what is undecided, and it is separated so that no reader mistakes one for the other.

Figure 24. Shipped, in review, and not started



The middle band matters as much as the first. Code that exists but has not been deployed is not a capability, and a document that counts it as one is misleading. The bottom band is what a prospective user should read first.

What is not ready, in detail

The network is not open for public sign-ups and should not be. Secret custody is not usable. The dedicated image has never booted. There is no tenant-facing monitoring on either network: the only cluster observability is a

pair of CLI tools that connect to each machine over SSH, which makes them operator-only. What a tenant can see is a health endpoint, a namespace status and per-deployment logs. And that health endpoint is structurally optimistic — a subsystem whose client was never constructed reports *unavailable*, which the aggregate counts as

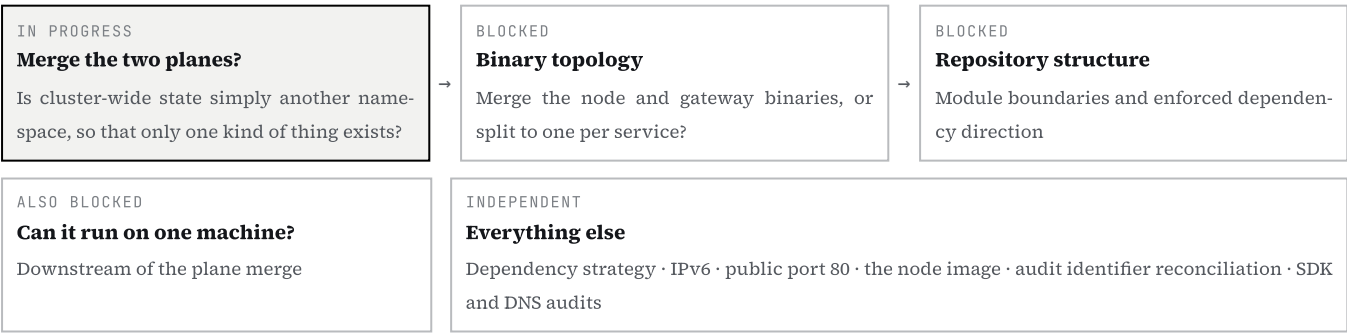
healthy, so a gateway with no cache connection at all still answers "healthy".

A security audit of our own system produced sixty-two findings that remain entirely untouched, and a later hardening sweep added more. Secret rotation is operationally impossible today: there is no key identifier in the encryption envelope, no re-encryption tool, and no rotation mechanism for the tunnel keys at all — so a key captured today stays valid indefinitely and there is no response available to a suspected compromise.

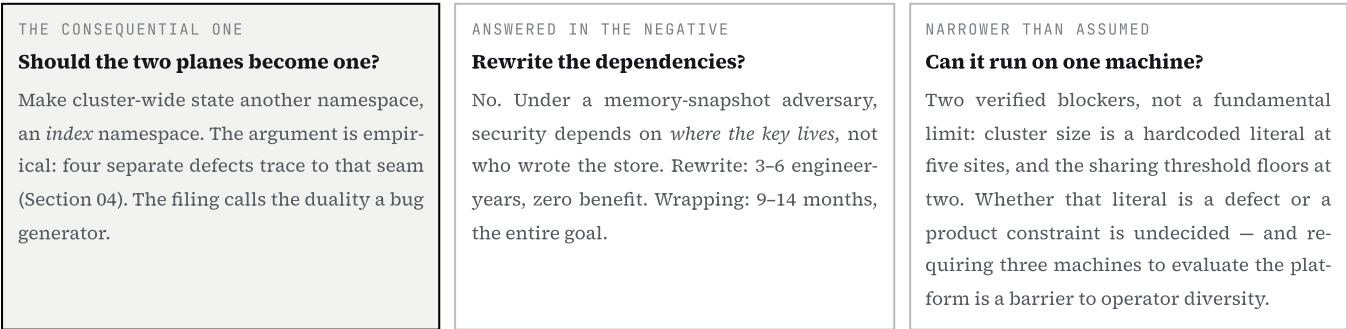
THE SCALE QUESTION, ASKED HONESTLY

The architecture has been exercised at the scale of a handful of nodes and a handful of tenants. Twenty namespaces per node is a designed ceiling; what has *not* been demonstrated is a network of hundreds of nodes with hundreds of tenants, operated by people who do not know each other. That is the real test of the thesis, and it has not been run.

Figure 25. What blocks what



You cannot lay out packages before you know the process boundaries. Starting at the wrong end of that chain means doing the work twice, which is why the gate exists.



Toward a first release

The draft definition on the table: a third-party developer can sign up, deploy an application with a database and a cus-

tom domain, and reach production without an operator touching anything — on a network where losing one node loses nothing, and where a provider snapshotting a disk gets ciphertext. Its value is that every clause is falsifiable.

22 OPEN DECISIONS AND DIRECTION

Thirteen architectural decisions are formally open, under a rule that no implementation tagged against them begins until an answer is recorded. None has been answered. Three are ordered, and the order is load-bearing. A fourth was settled by events: a dashboard and an agent-facing control interface were both dropped, so the CLI and the SDK are now the entire product surface.

#	Criterion	Today
01	Signup to a running application with a database and a custom domain, using only the CLI, docs and SDK	partial
02	Losing any single node loses no tenant data and no availability	partial
03	A disk snapshot yields ciphertext and no usable credential	fails
04	Every secret rotatable without downtime or data loss	fails
05	Nothing in the documentation is false	fails
06	A fleet upgrade is one command and cannot half-apply	partial

23 COLOPHON

Orama is a working decentralized application platform with a real tenant, a coherent architecture, an unusually candid audit of itself, and a substantial amount of unfinished work between here and something a stranger should trust.

The thesis — that the ordinary primitives of a cloud can be delivered from machines that many different people own, through an interface a developer already understands — is demonstrated at small scale. Whether it survives contact with hundreds of nodes and operators who do not know each other is the open question, and the only one that ultimately matters. We have tried to describe the system in terms that make that question answerable by someone outside the project, including by reporting the defects that would have been easier to omit.

Sources

Source tree	The Orama repository, read directly. Where it and the documentation disagreed, the source won.
Atlas	<i>Orama Network Atlas</i> — architecture reference, 51 pages. The complete technical mapping: binaries, systemd units, port arithmetic, the WASM sandbox, request paths.
Volume II	<i>The Hardening Playbook</i> — 90 findings across 8 tiers, each passed through adversarial review.
Volume III	<i>Rewrite or Replace?</i> — build-versus-buy analysis of five dependencies under the threat model of Part III.
Task tracker	The project board — open decisions, audits and defects, read in full.

Corrections carried in this edition

Three claims that appear in our own earlier documentation are corrected here rather than repeated: that the cluster database enforces HTTP authentication (it does not — Section 06); that secret custody is a usable service (Section 12); and that the dedicated node image is operational (Section 20). Each was found by reading the source against the document.

WHY PERMISSIONLESS NODES ARE DELIBERATELY OUT OF SCOPE

Permissionless participation requires that a hostile node be unable to harm a tenant, and the gaps in Section 21 are why that is not yet true. Opening node operation first would not be decentralization; it would be an unguarded multi-tenant system with more participants. The order is: **make one tenant safe from another, then make the network safe from its own nodes, then open it.**

ON THIS DOCUMENT

It is a technical description of a working system and an honest statement of what it does not do. It is not an offering document, it does not describe a token, and it makes no financial claim of any kind. Part V describes open questions and is labelled as such; nothing in it is a commitment or a schedule.

Errors here are defects. Report them the way you would report a defect in the code — with the section number and what the source actually says.

Written by members of DeBros, who remain anonymous for the same reasons this system exists. First edition.